NEOSTATS

Intelligence. Innovation. Impact.

Candidate Assignment
Data Science Hackathon

Loan Application Refusal Prediction & Credit Decision Dashboard

1. Objective

Build an end-to-end machine learning solution that predicts whether a loan application will be **refused** by the lender, and present the analysis and predictions in a Power BI dashboard (or Tableau for Mac users). This assignment tests your ability to develop an end-to-end data science solution — from raw data to a validated model and a dashboard that a business user can understand.

Your submission must include:

- Data understanding, cleaning and conversion into a model-ready format
- Feature engineering with a short justification per feature
- Exploratory data analysis with at least 5 business insights
- A Power BI (or Tableau) data model and dashboard
- A trained and validated model (train / validation / test + out-of-time)
- A clear strategy for class imbalance and data leakage
- Model evaluation KPIs and a clear reason for the final model choice
- A short presentation (PDF) with output screenshots

Use of AI tools: you are encouraged to use AI tools to help build the model and code, and to develop the dashboard.

Dashboard tool: use **Power BI Desktop** if you have a Windows laptop. Power BI Desktop does not run on Mac, so **Mac users can use Tableau** (Tableau Desktop or Tableau Public) instead. The same dashboard requirements apply to both tools.

What we are looking for: how well you understand the end-to-end process and how fully you cover the expectations of this use case. Visual polish, or going very deep in any one area, matters less.

2. Business Context

A large share of loan applications that reach a decision are refused. Every refused application still consumes underwriting time and leaves the customer with a poor experience. A model that predicts refusal at the point of application lets the lender:

- Pre-screen applications and route likely refusals to faster review
- Reduce wasted underwriting effort and turnaround time
- Guide customers to a loan amount, term or product they can get
- Understand which applicant and product traits drive refusals
- Give credit managers a dashboard they can read without Python
- Prioritise review effort using simple risk bands

3. Understanding the Data and What You Are Predicting

XYZ is a company that lends money to people, for example to buy a phone or a TV, or as a cash loan. Every time a person asks for a loan, they fill in an **application**. The company then says **Yes (Approved)** or **No (Refused)**. Over the years, the same person may apply many times. Your job is to predict, from the details of an application, whether the company will say No.

Where to Get the Data

Dataset link: https://github.com/neoYogesh/Neo_DS_Hackathon/releases/tag/data-v1

You do not need a GitHub account. To get the data:

1. Open the link above and scroll down to the **Assets** section.
2. Click the **.zip** file to download it. Ignore the two *Source code* files; they are not needed.
3. Unzip the files (on Windows: right-click → **Extract All**; on Mac: double-click).
4. Create your project folder (see the code structure in Section 9) and put the CSV files in its `data/` folder.
5. Load them in Python, for example `pd.read_csv("data/previous_application.csv")`. pandas can also read a zip directly: `pd.read_csv("previous_application.zip")`.

Tip: previous_application.csv is large (about 390 MB, 1.6 million rows). Download it before the hackathon starts, and close other programs if your laptop has less than 8 GB of RAM.

The Files You Will Use

File	What one row means	How to join
`previous_application.csv`	**One loan application** and the company's decision on it. This is your main table.	`SK_ID_PREV` = ID of the application
`application_train.csv`	**One customer**: age, income, job, family, housing and credit scores.	`SK_ID_CURR` = ID of the customer (also in the main table)
`columns_description.csv`	What each column means (the data dictionary).	—

One customer can have many applications, so join the customer details onto each application using `SK_ID_CURR`.

What You Are Predicting: REFUSED

Question: *Will the company say No to this loan application?* The answer is already recorded in the column `NAME_CONTRACT_STATUS`. You turn it into a 0/1 column:

Value in NAME_CONTRACT_STATUS	What it means in simple words	What you do
Approved	The company said Yes	Keep → **REFUSED = 0**
Refused	The company said No	Keep → **REFUSED = 1**
Canceled	The customer cancelled before a decision was made	Remove the row
Unused offer	The company said Yes, but the customer did not take the loan	Remove the row

We remove *Canceled* and *Unused offer* because in those cases the customer, not the company, decided what happened. In your notebook, write one or two lines on why this makes sense.

```
df = df[df["NAME_CONTRACT_STATUS"].isin(["Approved", "Refused"])]
df["REFUSED"] = (df["NAME_CONTRACT_STATUS"] == "Refused").astype(int)
```

Check the refusal rate yourself. After building the target, report the actual refusal rate in your cleaned data (the share of rows with REFUSED = 1). Comment on how much class imbalance this creates, and explain how you will handle it in Part 4.

How Time Works in This Data

Real dates are hidden. Instead, each date is a **negative number of days** counted back from the customer's latest application. In `DAYS_DECISION` (the day the company made its decision):

- `-30` means the decision was made 30 days before; `-365` means about 1 year before
- The more negative the number, the older the application
- Ages work the same way: `DAYS_BIRTH = -12000` means the customer was about 33 years old (12000 ÷ 365)

How to Split the Data

A model is built on the past but used on the future. So first set aside the **most recent 12 months** as an out-of-time (OOT) set, then split the older data into train, validation and test. Think of it as practising on past exam papers and then sitting this year's exam.

Set	Data used	Purpose
Train	70% of older data (`DAYS_DECISION ≤ -365`)	Fit preprocessing and model
Validation	15% of older data	Model and threshold selection
Test	15% of older data	Final check, used once

Set	Data used	Purpose
Out-of-time (OOT)	Most recent 12 months (`DAYS_DECISION > -365`)	Check stability on newer data

- Split train, validation and test with **stratification on the target**, so each set has the same refusal rate
- Build and tune on **train**, compare and choose on **validation**, and score **test** and **OOT** only at the end

Watch Out: Columns That Give Away the Answer (Data Leakage)

Some columns are only filled in **after** the company has decided. For example, the date the money was paid to the customer only exists if the loan was approved. If your model uses such a column, it is like seeing the answer key during an exam: the score looks great, but the model is useless for a new application. Find these columns, remove them, and list each one in your notebook with the reason.

Tip: if your model scores close to 100%, that is a warning sign that you have probably kept one of these columns.

Odd Values You Will Meet

Value	Where	What it means
`365243`	Some `DAYS_*` columns	A placeholder for “no date”, not a real number of days
`XNA`	Text columns	Not available (missing)
`XAP`	Text columns	Not applicable (the question did not apply to this application)

4. Assignment Scope

#	Module	What to Build
1	Data Understanding & Preparation	Profile the data, fix quality issues, join the tables, build the target, check the refusal rate and remove leakage columns.
2	Feature Engineering	Create meaningful ratio, flag, binned and history features; encode and impute.
3	EDA & Data Model	Analyse refusal drivers in Python; build a star-schema data model in Power BI or Tableau.
4	Model Development	Train a baseline and at least one stronger model; handle class imbalance.
5	Evaluation, Validation & Selection	Compare models with KPIs on validation, choose the final model and threshold, then score test and OOT.
6	Dashboard (Power BI or Tableau)	EDA page, model performance page, predictions & feature importance page.

5. Functional Requirements

Each part lists the **minimum** required to score and an optional **stretch** goal. Finish every minimum before attempting a stretch goal — an end-to-end solution scores higher than one excellent part.

Part 1: Data Understanding & Preparation

- Load the files; check shapes, data types, missing values and duplicates
- Join client attributes to applications on `SK_ID_CURR`
- Build the REFUSED target and report the **actual refusal rate** in the cleaned data
- Treat known data issues, e.g. the placeholder `365243` in `DAYS_*` columns and `XNA` / `XAP` category codes
- Remove post-decision (leakage) columns and document each one
- Convert `DAYS_*` columns to readable units (age in years, employment in years)
- Save a clean, model-ready table (`.parquet` or `.csv`)

Deliverables:

Data quality summary, refusal rate and imbalance comment, feature categorisation (demographic, financial, product, channel, history) and leakage list.

Stretch: a data quality table with % missing and the action taken per column.

Part 2: Feature Engineering

Create at least **8 engineered features** and explain each in one line. Examples:

- `AMT_APPLICATION / AMT_INCOME_TOTAL` — requested amount vs income
- `AMT_APPLICATION / AMT_GOODS_PRICE` — share of the goods price financed
- Age band, employment-length band, income band
- Weekend application flag, application hour band
- Count of missing external scores; mean of `EXT_SOURCE_1/2/3`

Encode categorical columns, impute missing values and justify your choices.

Stretch: point-in-time history features — for each application, the number of the client's **earlier** applications and how many were refused, using only rows with a smaller `DAYS_DECISION`.

Part 3: EDA & Data Model

- Univariate and bivariate analysis of refusal rate against key features
- Refusal rate by product type, channel, loan purpose, client type, income band and age band
- Trend of applications and refusal rate over time (derived from `DAYS_DECISION`)
- At least **5 business insights**, each stated as a finding with a number (e.g. “Refusal rate for X is 2.3× that of Y”)
- In Power BI or Tableau, build a **star-schema data model**: a fact table of applications, dimension tables (client, product, channel, date), relationships and at least **4 measures** (DAX measures in Power BI, calculated fields in Tableau), e.g. Applications, Refusal Rate, Avg Requested Amount, Avg Predicted Probability

Deliverables:

EDA notebook with charts, and the EDA page of the dashboard.

Part 4: Model Development

- Train a simple, interpretable baseline (e.g. Logistic Regression)
- Train at least one stronger model (e.g. Random Forest, XGBoost, LightGBM)
- Handle class imbalance with at least one justified method: class weights, `scale_pos_weight`, resampling (SMOTE / undersampling — on training data only) or threshold tuning
- Use a reproducible pipeline (fixed random seed; preprocessing fitted on training data only)
- Save the final model (`.pkl` / `.joblib`)

Stretch: light hyperparameter tuning with cross-validation; probability calibration.

Part 5: Evaluation, Validation & Selection

- Use the train / validation / test / OOT split described in Section 3
- Report ROC-AUC, PR-AUC, precision, recall, F1 and the confusion matrix
- Compare all models in one KPI table on the validation set and explain **why you chose the final model**
- Choose a probability threshold on validation and explain the trade-off between precision and recall
- Explain why accuracy is not a suitable main metric here
- Score the test set once, then the OOT set, and compare the two; explain any drop
- Convert probabilities into a **risk band** (Low / Medium / High) and show the refusal rate per band
- Export **feature importance** from the final model (global view) for the dashboard

Stretch: KS statistic, Gini, decile (lift) table, and a stability check (PSI) between development and OOT score distributions.

Part 6: Dashboard (Power BI or Tableau)

Build a dashboard of at least three pages:

1. **EDA overview** — KPIs (applications, refusal rate), refusal rate by key dimensions, trend over time, slicers
2. **Model performance** — metrics for test vs OOT, confusion matrix, refusal rate by risk band
3. **Predictions & feature importance** — application-level table with probability and risk band; global feature importance from the final model

The dashboard should tell a story a credit manager can follow without help.

6. Recommended Technical Stack

Use the stack below. If you choose something different, briefly explain your reasoning in the README.

Component	Recommended Choice
Language	Python 3.10+ (Jupyter notebooks)
Data	pandas, numpy
Modelling	scikit-learn, LightGBM or XGBoost, imbalanced-learn (optional)
EDA visualisation	matplotlib, seaborn or plotly
Dashboard	Power BI Desktop (Windows) or Tableau Desktop / Tableau Public (Mac)

7. Expected Deliverables

Code & outputs (GitHub repository or zip):

- Data preparation and feature engineering notebook
- EDA notebook with charts and written insights
- Model training, evaluation, validation and OOT notebook
- Saved model artefact and CSV files used by the dashboard
- Dashboard file (`.pbix` for Power BI or `.twbx` for Tableau)
- `requirements.txt` and `README.md`

Documentation (in README.md):

- How to run the notebooks in order
- Target definition, actual refusal rate, leakage columns removed and why
- Split strategy (train / validation / test / OOT) and class imbalance approach
- KPI comparison table, final model choice and threshold
- Data model design (tables, relationships, measures)
- Known limitations and possible improvements

8. Evaluation Criteria

Your submission will be scored across the following areas. We are mainly assessing how well you understand and cover the end-to-end process, not visual polish or depth in any one area.

Evaluation Area	Weightage	Covers
Model development and validation	25%	Splits, OOT, class imbalance handling
Model evaluation KPIs and selection	15%	KPIs used, how models were compared, why the final model was chosen, threshold
Data understanding, cleaning and leakage handling	15%	Data quality checks, joins, target and refusal rate, leakage removal
Feature engineering	15%	Engineered features and their justification
EDA insights and data model	15%	Insights with numbers, star schema, measures
Prediction dashboard and storytelling	10%	Three pages, predictions, risk bands, feature importance
Code quality, README and presentation	5%	Clean notebooks, clear README, slides
Total	**100%**	

9. What We Look For

- Correct target construction and no data leakage
- Preprocessing fitted only on training data

- A sensible, justified approach to class imbalance
- Clear KPI comparison and reasoning for the final model and threshold
- Honest comparison of test vs OOT performance, with an explanation of any drop
- Insights stated with numbers, not just charts
- A working end-to-end flow: raw data → model → predictions → dashboard
- Clean, readable notebooks that run top to bottom

A simple, correct and well-explained solution will score higher than a complex one with leakage or no validation.

Code Structure to be followed:

```
refusal_prediction/
├── data/                                  Raw files (not committed to git)
├── notebooks/
│   ├── 01_data_preparation.ipynb          Cleaning, leakage removal, features
│   ├── 02_eda.ipynb                       EDA and business insights
│   └── 03_modelling_validation.ipynb      Training, KPIs, test and OOT
├── outputs/                               Tables exported for the dashboard
│   ├── fact_applications.csv
│   ├── dim_client.csv
│   ├── predictions.csv                    id, probability, risk band, split
│   ├── feature_importance.csv             feature, importance (final model)
│   └── model_metrics.csv                  KPIs by model and by split
├── models/
│   └── final_model.joblib                 Saved model artefact
├── dashboard/
│   └── refusal_dashboard.pbix             Power BI (or .twbx for Tableau)
├── documents/
│   └── project_presentation.pdf           Project presentation (PDF)
├── requirements.txt                       Python dependencies
└── README.md                              Project documentation
```

10. Submission Instructions

- GitHub repository link or zipped source code (no raw data, no virtual environment folders)
- The dashboard file (`.pbix` or `.twbx`) in `/dashboard` and screenshots of every dashboard page
- README with step-by-step run instructions and a brief note on major design decisions
- A 5–7 slide presentation saved as PDF in `/documents`: problem and target, refusal rate, key EDA insights, model comparison and final choice, test vs OOT results, feature importance, dashboard screenshots, limitations
- Be ready for a 5-minute walkthrough of your solution
- A Microsoft Form will be shared for final submission